Домашнее задание по курсу "Введение в тензорные компиляторы"

Кодогенерация

Шелонин Арсений Б01-411
shelonin.ak@phystech.edu

Содержание

1 Описание задачи **2**

2 Легализация и выбор **5**

3 Разрушение SSA формы **7**

4 Распределение регистров **9**

5 Планирование инструкций блока `loop2_header` **13**

1 Описание задачи

В прошлом задании было необходимо перевести код на C в Generic-IR SSA форму и построить CFG.

Исходный код на C

```c
int function1(int* a, int n) {
    int result = 0;
    for (int i = 0; i < n; ++i) {
        int x = 5;
        for (int j = 0; j < n; ++j) {
            if (j % 2 == 0)
                continue;
            result += 2 * 3 * a[j] * x - i - 2;
        }
    }

    return result;
}
```

SSA IR (optimised)

```
int function1(int* a, int n) {
  loop1_preheader:
    result0 = 0;
    i0 = 0;
    loop1_neg_cnd = (i0 >= n);
    br loop1_neg_cnd, exit, loop1_header;

  loop1_header:
    result1 = phi(result0, result5);
    i1 = phi(i0, i2);
    j0 = 1;
    loop2_cnd0 = (j0 < n);
    br loop2_cnd0, loop2_header, loop1_latch;

  loop2_header:
    result2 = phi(result1, result3);
```

```
    j1 = phi(j0, j2);
    addr0 = a + j1;
    val0 = *addr0;
    tmp0 = 30 * val0;
    tmp1 = tmp0 - i1;
    tmp2 = tmp1 - 2;
    result3 = result2 + tmp2;
    br loop2_latch;

  loop2_latch:
    result3 = result3;
    j2 = j1 + 2;
    loop2_cnd1 = (j2 < n);
    br loop2_cnd1, loop2_header, loop1_latch;

  loop1_latch:
    result5 = phi(result1, result3);
    i2 = i1 + 1;
    loop1_cnd0 = (i2 < n);
    br loop1_cnd0, loop1_header, exit;

  exit:
    result6 = phi(result0, result5);
    return result6;
}
```

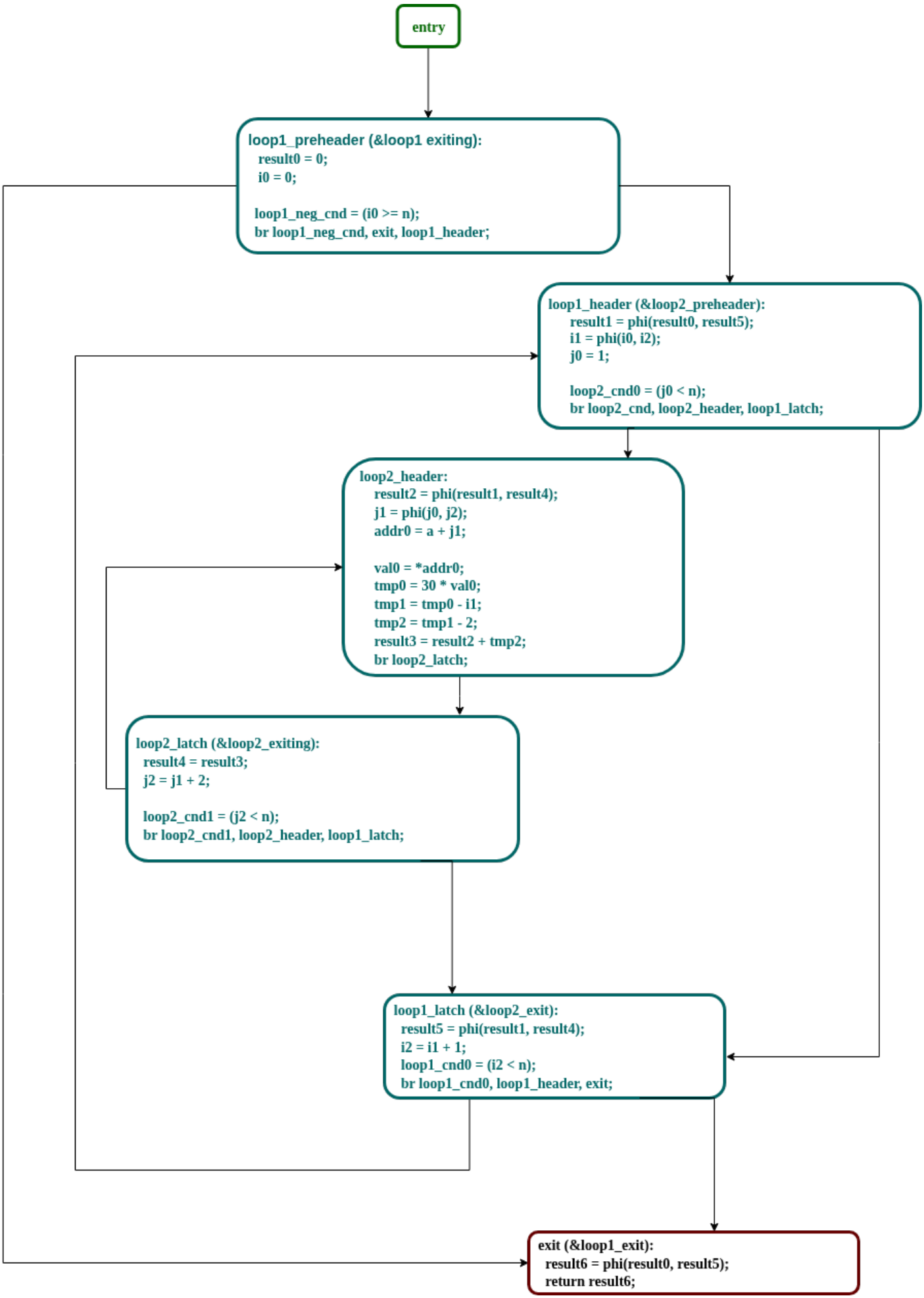

Рис. 1: SSA CFG (optimised)

Сейчас задача состоит в том, чтобы преобразовать Generic-IR код в ассемблер (будет использоваться ассемблер x86-64).

2 Легализация и выбор

Архитектура x86-64 требует двухадресного формата инструкций, в то время как Generic-IR имеет трехадресную архитектуру.

В коде присутствуют многочисленные операции формата `addr1 = addr0 + int_shift0`. Изначально компилятор переводит их в поступательное вычисление адреса с учетом размера типа `int` и сложения с `addr0`, но после процесса выбора инструкций происходит матч с инструкций `lea` и свёртка в неё.

Аналогичным образом сворачивается условный переход.

Операция	**Generic IR**	**x86-64 legalised**
Трехадресный код, не поддерживаемый ассемблером	`tmp3 = tmp1 <op> tmp2`	`mov tmp3, tmp1` `<asm_op> tmp3, tmp2`
Адресная арифметика	`addr1 = addr0 + int_shift0`	`lea addr1, [addr0 + int_shift0*4]`

Условный переход	`cnd0 = (j0 <cnd> j1)`
	`br cnd0, bb0, bb1`
		`cmp j0, j1`
		`jump<cnd> bb0`
		`jmp bb1`

SSA IR (legalised)

```
int function1(int* a, int n) {
loop1_preheader:
  mov result0, 0
  mov i0, 0
  cmp i0, n
  jge exit
  jmp loop1_header

loop1_header:
  result1 = phi(result0, result5)
  i1 = phi(i0, i2)
  mov j0, 1
  cmp j0, n
  jl loop2_header
  jmp loop1_latch
```

```

loop2_header:
  result2 = phi(result1, result3)
  j1 = phi(j0, j2)

  lea addr0, [a + j1*4]
  mov val0, [addr0]

  mov tmp0, 30
  imul tmp0, val0

  mov tmp1, tmp0
  sub tmp1, i1

  mov tmp2, tmp1
  sub tmp2, 2

  mov result3, result2
  add result3, tmp2

  jmp loop2_latch

loop2_latch:
  mov result3, result3

  mov j2, j1
  add j2, 2

  cmp j2, n
  jl loop2_header
  jmp loop1_latch

loop1_latch:
  result5 = phi(result1, result3)

  mov i2, i1
  add i2, 1

  cmp i2, n
  jl loop1_header
  jmp exit
```

```

exit:
  result6 = phi(result0, result5)
  ret result6
}
```

3 Разрушение SSA формы

IR (no SSA)

```
int function1(int* a, int n) {
  loop1_preheader:
  mov result0, 0
  mov i0, 0

  mov result1, result0
  mov result6, result0
  mov i1, i0

  cmp i0, n
  jge exit
  jmp loop1_header

loop1_header:   // result1 <- phi(result0, result5)
                // i1      <- phi(i0, i2)
  mov j0, 1

  mov j1, j0
  mov result2, result1
  mov result5, result1

  cmp j0, n
  jl loop2_header
  jmp loop1_latch

loop2_header:
                // result2 <- phi(result1, result3)
                // j1      <- phi(j0,      j2)

  lea addr0, [a + j1*4]
```

```
  mov val0, [addr0]

  mov tmp0, 30
  imul tmp0, val0

  mov tmp1, tmp0
  sub tmp1, i1

  mov tmp2, tmp1
  sub tmp2, 2

  mov result3, result2
  add result3, tmp2

  jmp loop2_latch

loop2_latch:
  mov j2, j1
  add j2, 2

  mov result4, result3
  mov j1, j2
  mov result2, result4
  mov result5, result4

  cmp j2, n
  jl loop2_header
  jmp loop1_latch

loop1_latch:
                // result5 <- phi(result1, result4)

  mov i2, i1
  add i2, 1

  mov i1, i2
  mov result1, result5
  mov result6, result5

  cmp i2, n
  jl loop1_header
```

```
  jmp exit

exit:            // result6 <- phi(result0, result5)
  ret result6
}
```

4 Распределение регистров

Выберем самый нагруженный блок — loop2_header — и распределим в нём регистры линейным подходом.

```
loop2_header:                // a, i1, j1, result2 - already defined
  lea addr0, [a + j1*4]      // def addr0, use a, use j1
  mov val0, [addr0]          // def val0, use addr0

  mov tmp0, 30               // def tmp0
  imul tmp0, val0            // use tmp0, use val0, def tmp0

  mov tmp1, tmp0             // def tmp1, use tmp0
  sub tmp1, i1               // use tmp1, use i1, def tmp1

  mov tmp2, tmp1             // def tmp2, use tmp1
  sub tmp2, 2                // use tmp2, def tmp2

  mov result3, result2       // def result3, use result2
  add result3, tmp2          // use result3, use tmp2, def result3

  jmp loop2_latch
```

```
a: defined
i1: defined
j1: defined
result2: defined
addr0: (2, 3)
val0: (3, 6)
tmp0: (5, 8)
tmp1: (8, 9)
tmp2: (11, 15)
result3: (14, 15)
```

current	**live**	**old**
addr0	addr0	

```
a: rdi
i1: rbx
j1: rcx
result2: rax
addr0: r8
val0: (3, 6)
tmp0: (5, 8)
tmp1: (8, 9)
tmp2: (11, 15)
result3: (14, 15)
```

```
a: defined
i1: defined
j1: defined
result2: defined
addr0: (2, 3)
val0: (3, 6)
tmp0: (5, 8)
tmp1: (8, 9)
tmp2: (11, 15)
result3: (14, 15)
```

current	**live**	**old**
val0	val0	addr0

```
a: rdi
i1: rbx
j1: rcx
result2: rax
addr0: r8
val0: r8
tmp0: (5, 8)
tmp1: (8, 9)
tmp2: (11, 15)
result3: (14, 15)
```

```
a: defined
i1: defined
j1: defined
result2: defined
addr0: (2, 3)
val0: (3, 6)
tmp0: (5, 8)
tmp1: (8, 9)
tmp2: (11, 15)
result3: (14, 15)
```

current	**live**	**old**
tmp0	val0	addr0
	tmp0	

```
a: rdi
i1: rbx
j1: rcx
result2: rax
addr0: r8
val0: r8
tmp0: r9
tmp1: (8, 9)
tmp2: (11, 15)
result3: (14, 15)
```

```
a: defined
i1: defined
j1: defined
result2: defined
addr0: (2, 3)
val0: (3, 6)
tmp0: (5, 8)
tmp1: (8, 9)
tmp2: (11, 15)
result3: (14, 15)
```

current	**live**	**old**
tmp1	tmp1	addr0
		val0
		tmp0

```
a: rdi
i1: rbx
j1: rcx
result2: rax
addr0: r8
val0: r8
tmp0: r9
tmp1: r8
tmp2: (11, 15)
result3: (14, 15)
```

```
a: defined
i1: defined
j1: defined
result2: defined
addr0: (2, 3)
val0: (3, 6)
tmp0: (5, 8)
tmp1: (8, 9)
tmp2: (11, 15)
result3: (14, 15)
```

current	**live**	**old**
tmp2	tmp2	addr0
		val0
		tmp0
		tmp1

```
a: rdi
i1: rbx
j1: rcx
result2: rax
addr0: r8
val0: r8
tmp0: r9
tmp1: r8
tmp2: r8
result3: (14, 15)
```

```
a: defined
i1: defined
j1: defined
result2: defined
addr0: (2, 3)
val0: (3, 6)
tmp0: (5, 8)
tmp1: (8, 9)
tmp2: (11, 15)
result3: (14, 15)
```

current	**live**	**old**
result3	tmp2	addr0
	result3	val0
		tmp0
		tmp1
		result2

```
a: rdi
i1: rbx
j1: rcx
result2: rax
addr0: r8
val0: r8
tmp0: r9
tmp1: r8
tmp2: r8
result3: rax
```

Итоговый ассемблерный код Важно обратить внимание на интерфейс вызова функции System V AMD64 ABI. Аргумент 1 передается регистром `rdi`, аргумент 2 - регистром `rsi`. Callee-saved регистры - `rbx, rbp, r12-r15`, то есть мы обязаны сохранить `rbx`.

```
function1:
push rbx
loop1_preheader:
  mov rax, 0                # result0 = 0
  mov rbx, 0                # i0 = 0

  cmp rbx, rsi              # n -> rsi
  jge exit
  jmp loop1_header

loop1_header:
  mov rcx, 1                # j0 = 1

  cmp rcx, rsi              # n -> rsi
  jl loop2_header
  jmp loop1_latch

loop2_header:
  lea r8, [rdi + rcx*4]     # addr0 = a + j1*4
  movsxd r8, dword ptr [r8] # val0 = *addr0

  mov r9, 30                # tmp0 = 30
  imul r9, r8               # tmp0 *= val0

  mov r8, r9                # tmp1 = tmp0 (tmp1: r8, tmp0: r9)
  sub r8, rbx               # tmp1 -= i1

  sub r8, 2                 # tmp2 = tmp1 - 2

  add rax, r8               # result3 (rax) += tmp2

  jmp loop2_latch

loop2_latch:
  add rcx, 2                # j2 = j1 + 2

  cmp rcx, rsi
```

```
  jl loop2_header
  jmp loop1_latch

loop1_latch:
  add rbx, 1                # i2 = i1 + 1

  cmp rbx, rsi
  jl loop1_header
  jmp exit

exit:
  pop rbx
  ret                       # ret rax
```

5 Планирование инструкций блока `loop2_header`

Определим времена раннего и позднего планирования с учётом времени выполнения инструкций:

инструкция	**время исполнения (такты)**
lea	1
mov	1
imul	3
add	1
sub	1

- Ипользуется **алгоритм списочного планирования (list sheduling):**

  1. Используется два списка:

     Список готовых: могут начать выполняться прямо сейчас.

     Список активных — вызвыаны в предыдущем такте, но всё ещё не готовые

  2. Из списка готовых операций выбираем те, которые будем исполнять на данном такте, и помещаем их в список активных.

  3. Смещаем счётчик такта на единицу.

  4. Если операция из списка активных завершилась — она спланирована.

5. Если в узле больше нет операций — завершаем алгоритм.
6. Среди потомков спланированных операций в графе зависимостей ищем такие, аргументы которых уже вычислены, и помещаем в список готовых.
7. Переходим к п. 1.

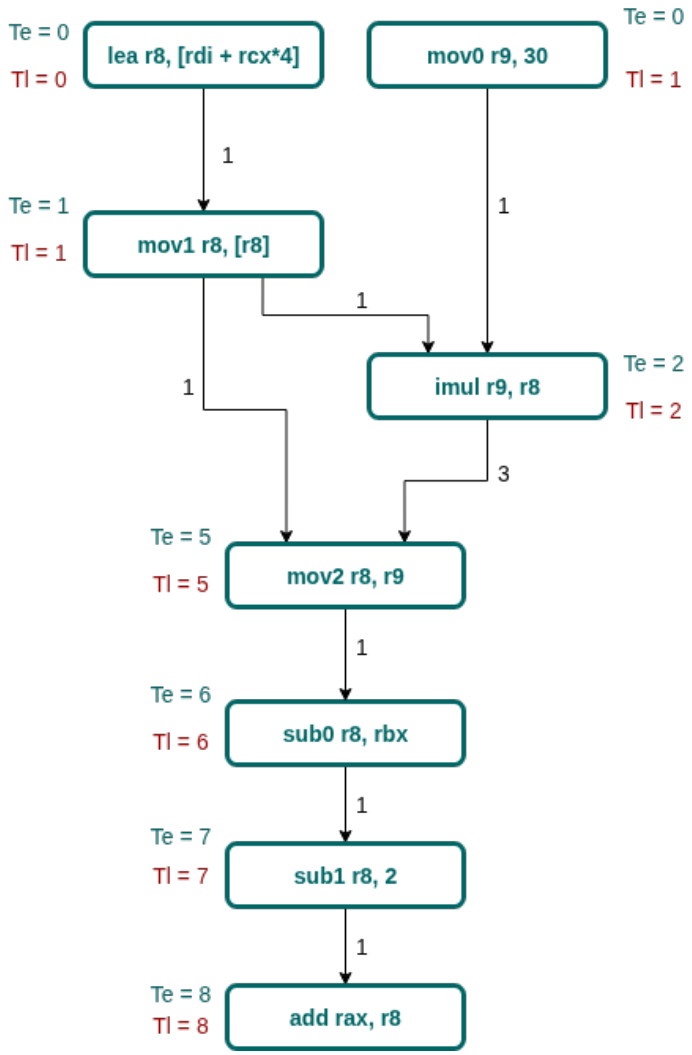

Рис. 2: T_early_late

такт	**m1**	**m2**	**готовые**	**запланированные (активные)**
0	lea	mov0	lea, mov0	lea, mov0
1	mov1	–	mov1	mov1
2	imul	–	imul	imul
3	imul	–	–	imul
4	imul	–	–	imul
5	mov2	–	mov2	mov2
6	sub0	–	sub0	sub0
7	sub1	–	sub1	sub1
8	add0	–	add0	add0

Заметно, как второй АЛУ простаивает практически всё время работы. Это связано с истиными RAW-зависимостями в блоке, что подтверждает практически линейный вид графа выше.